



GROUP ASSIGNMENT

CT111-3-2-COMT

COMPUTING THEORY

APD2F2411CS(CYB)

HAND OUT DATE: 4TH MAY 2025

HAND IN DATE: 25TH JULY 2025

GROUP: P

No.	Student Name	TP number
1	HEMANTHRAJAH A/L N KRISHNARAJAH	TP070844
2	ANUSHA DEVY A/P BRAHMATARAN	TP071154
3	DARMIK ISHWAR A/L RAVI	TP070071
4	ADAM JEROME JOSEPH	TP071194
5	SAILESS RAJ A/L RAJENDRAN	TP070996

Contents

Part A	3
1.1 Introduction	3
1.1.1 Design a NFA for Syntax of Chosen Construct.....	4
1.1.2 Quintuple and State Transition Table for NFA.....	6
1.1.3 Regular Expressions	8
Conclusion Part A	15
Part B	16
1.1 Best-Case Scenario	17
1.2 Worst-Case Scenario	17
2.0 Computability	18
2.1 Complete Computability.....	18
2.2 Mechanism Computability.....	18
2.3 Deterministic Computability	19
Conclusion Part B	20
References.....	21

Part A

1.1 Introduction

Python is an interpreted, high-level, object-oriented programming language known for its dynamic semantics. Its built-in data structures, coupled with dynamic typing and dynamic binding, make it ideal for Rapid Application Development (RAD) and for connecting existing software components as a scripting or "glue" language.

In programming, iteration involves repeatedly executing a set of instructions either for a specific number of times or until a certain condition is met. This project utilizes both For Loops and Do...While Loops for iterative processes.

The provided code snippet focuses on updating restaurant stock. Currently, it allows users to specify the type and quantity of inventory. As this is a fundamental update function, various solutions are available to users. The project documentation clearly defines the quintuple and state transition table for the Non-deterministic Finite Automaton (NFA), explicitly outlining its finite-state machine. For question 3, the regular expression for this NFA is provided, which serves as a pattern describing the criteria for strings that match it.

1.1.1 Design a NFA for Syntax of Chosen Construct.

```
def basic_inventory():  
    restaurant_item = input("\n\tPlease Enter The Restaurant Item: ")  
    while restaurant_item != "Fried Noodles" and restaurant_item != "Cheese pizza" and restaurant_item != "Flat bread" and restaurant_item != "Chicken burger":  
        print("\nThere is no record for Restaurant Item!")  
        restaurant_item = input("\n\tPlease Enter The Restaurant Item: ")  
    quantity = int(input("\n\tPlease Enter The Quantity : "))  
    print("\nQuantity saved!")  
    print("Exit")
```

Figure 1: Code snippet

In the given Python code, the message “Quantity saved” will only appear once two specific conditions are met successfully. The first condition requires the user to select the correct restaurant item. If an incorrect item is entered, the program responds with an error message: “There is no record for Restaurant Item!” prompting the user to try again with the correct item. Once the correct item is entered, the user is then prompted to input the quantity for that item. After providing the quantity, they can save the data. At this point, the loop concludes.

However, if the user wants to add more quantities, the same process will repeat from the beginning. This entire flow operates within a “do...while” loop structure.

For the code to work as intended and produce the final result, both conditions must be fulfilled in the correct sequence, and the proper menu option must be selected to reach the output. Failing to meet these criteria will result in the process being terminated without generating the final output, thus not reaching the accept state.

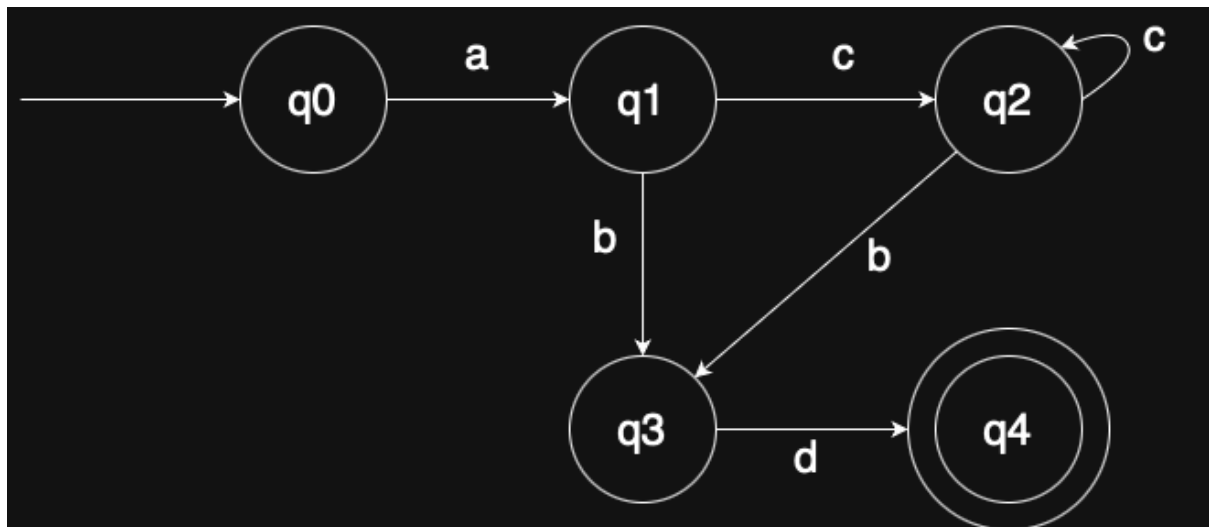


Figure 2: NFA diagram

This NFA models the process of a user interacting with a system to order a restaurant item, focusing on the validation and saving of the input. The process begins at an initial state (q0) where the user is prompted to enter a restaurant item ('a'). Upon entering an item, the system validates it. If the entered item is invalid (i.e., not "Fried Noodles," "Cheese pizza," "Flat bread," or "Chicken burger"), the NFA transitions to a state (q2) that represents an error or re-prompt, indicated by 'c'. From this state, the user is repeatedly prompted for input (represented by the self-loop 'c') until a valid item is provided. Once a valid item is entered, either directly from the initial prompt ('b' from q1) or after several invalid attempts ('b' from q2), the NFA moves to a state (q3) where the quantity is requested, and the input is considered validated. Finally, the process concludes when the changes are "saved" ('d'), transitioning the NFA to its final accepting state (q4), signifying a successful order entry.

1.1.2 Quintuple and State Transition Table for NFA

a) Quintuple

q	{q0, q1, q2, q3, q4}
q0	q0
F	q4
Σ	{a, b, c, d}

*Table 13: Quintuple Table*b) State Transition Table

Input \ State	a	b	c	d
q0	q1	q0	q0	q0
q1	q0	q3	q2	q0
q2	q0	q3	q2	q0
q3	q0	q0	q0	q4
q4	q0	q0	q0	q0

Table 24: State Transition Table

1. In the initial row, the current state is q_0 . If the character received is 'a', the machine advances to q_1 . However, if the input is 'b', 'c', or 'd', the state is unchanged at q_0 .
2. The second line represents the behaviour in state q_1 . If the incoming symbol is 'a' or 'd', the automaton returns to q_0 . When the input is 'b', it moves forward to q_3 , and if it encounters 'c', it transitions to q_2 .
3. In the third line, where the system is in q_2 , an input of 'b' causes it to jump to q_3 . If the symbol is 'a' or 'd', it reverts to q_0 . An input of 'c' keeps the automaton in q_2 .
4. The fourth line covers the case when the active state is q_3 . If the machine reads 'a', 'b', or 'c', it resets to q_0 . Only when the input is 'd' does it shift to q_4 .
5. Lastly, in the fifth row, representing state q_4 , any input whether 'a', 'b', 'c', or 'd', will cause the machine to return to the starting state q_0 , since q_4 is a terminal or final state.

1.1.3 Regular Expressions

Step 1:

This is the main NFA diagram that we made. To proceed on to the subsequent phase, we want to attach an addition start state and accept state. Afterwards, in each step, state q_4 , q_3 , q_2 , q_1 , and q_0 are eliminated.

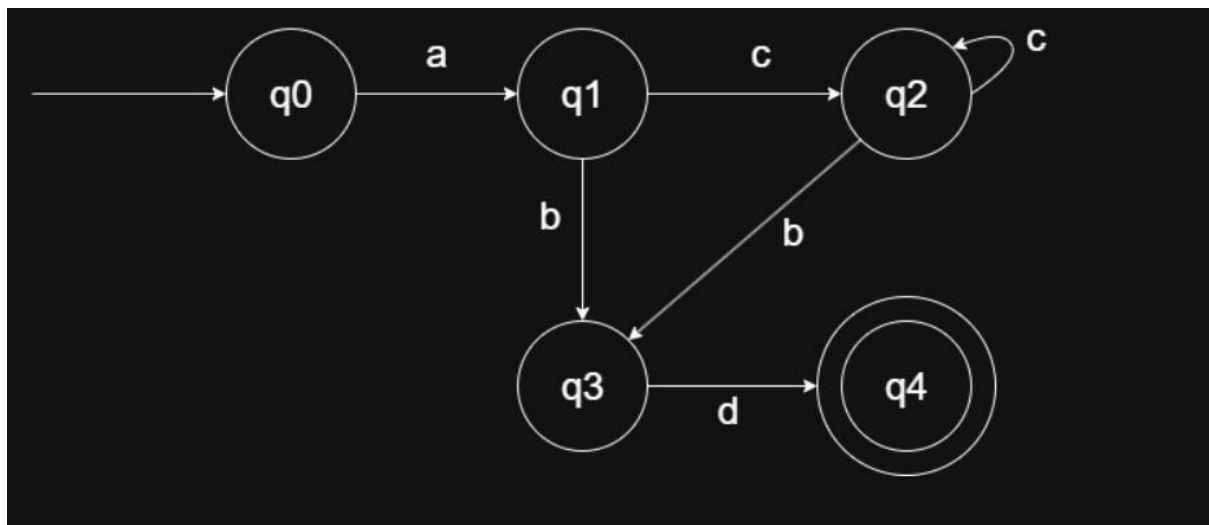


Figure 3: NFA Diagram

Step 2:

We incorporated a start state, designated "S," in Figure 1.2. The first start state of this NFA, q0, will be stretched through an epsilon traversal from this start state. Correspondingly, we created state "F" with an epsilon traversal that travels from the q4 to the accept state.

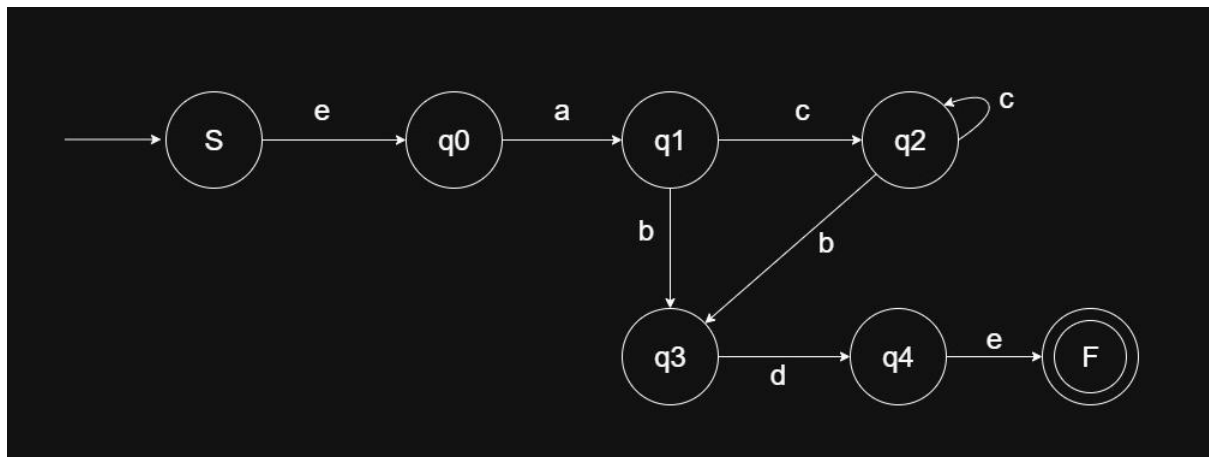


Figure 4: NFA Diagram with a start state designated "S"

Step 3:

We are eliminating the q_4 in Figure 1.3, which demonstrates the arrows coming in and coming out from the specific state. The In and Out of the State Q_4 is presented in the table below.

Incoming	Outgoing
q_3	F

Table 3: Removal of q_4

Consequently, q_3 is the state that inserts q_4 , and F is the state that let out q_4 . At this period, we struggled to pretend state q_4 vanishing by traveling straight from state q_3 to state "F." The last transition will be "d" from state q_3 to state "F" since state q_3 first enters state q_4 with a transition of "d," and state q_4 to state "F" has an empty transition. State Q_4 is therefore detached.

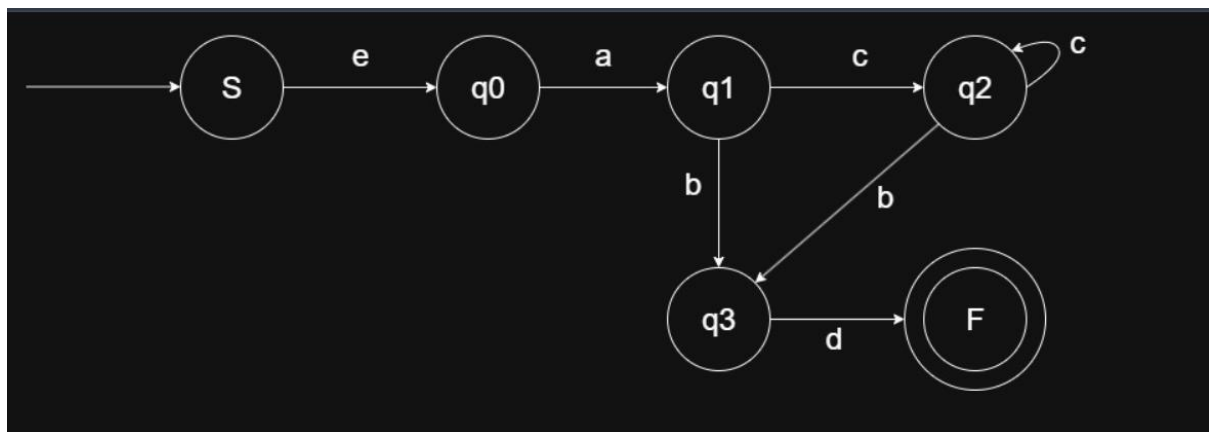


Figure 5: NFA Diagram with removal of q_4

Step 4:

Q3, Q2, Q0, and Q1 are similar for the removal of state q3. For the consistent steps, tables are delivered for each condition of elimination.

The new transition will be emphasised when there is a self-loop. State q2 had a self-loop of "c," as shown in Figure 1.4 below, and when state q2 was detached, "c" was introduced on a innovative changeover from state q1 to state "F."

Incoming	Outgoing
q1,q2	F

Table 4: Removal of q3

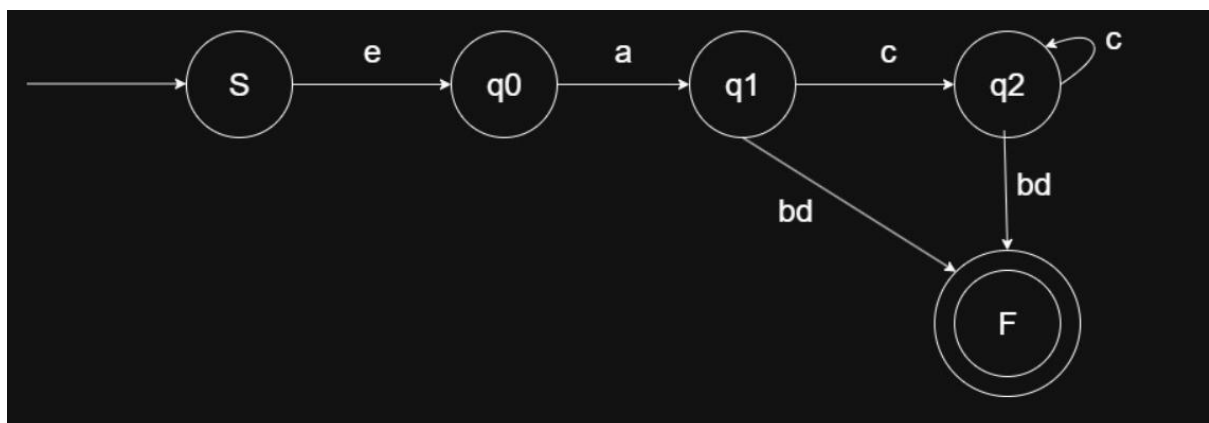
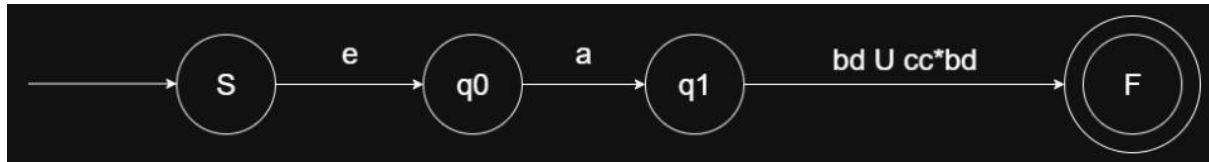


Figure 6: NFA Diagram with removal of q3

Step 5:

Incoming	Outgoing
q1	F

Table 5: Removal of q2*Figure 7: NFA Diagram with removal of q2*

We are now eliminating the q2 state. Table 5 demonstrates the incoming and outgoing arrows from the specific state.

When q2 is removed, we look at the paths that previously passed through it. The self-loop on q2, labeled "c", needs to be accounted for. This means any path that went through q2 could have taken the "c" loop any number of times.

Combining the transition from q1 to q2 labeled "c" with the self-loop "c" on q2 and the transition from q2 to F labeled "bd", we form a new path from q1 to F. This new path represents "c" followed by any number of "c"s (due to the self-loop) and then "bd". This is combined with the existing "bd" path from q1 to F (from the removal of q3 in the previous step). Therefore, the new transition from q1 to F becomes "bd U cc*bd", as shown in Figure 7.

This process simplifies the NFA by bypassing q2.

Step 6:

We are now eliminating the q_0 state. Table 6 demonstrates the incoming and outgoing arrows from the specific state.

Consequently, S is the state that inserts q_0 , and q_1 is the state that lets out q_0 . At this period, we simplify the NFA by traveling straight from state S to state q_1 . The transition from S to q_0 was labeled "e", and the transition from q_0 to q_1 was labeled "a". When q_0 is removed, these two transitions are combined into a single new transition directly from S to q_1 , which becomes "ea". State q_0 is therefore detached. This leads to the NFA diagram shown in Figure 8.

Incoming	Outgoing
S	q_1

Table 6: Removal of q_0



Figure 8: NFA Diagram with removal of q_0

Step 7:

We are now eliminating the q_1 state, which is the final intermediate state to be removed. Table 7 demonstrates the incoming and outgoing arrows from this specific state.

Consequently, S is the state that leads into q_1 , and F is the state that receives transitions from q_1 . The incoming transition to q_1 was from S, labeled "ea" (from the removal of q_0). The outgoing transition from q_1 to F was labeled " $bd \cup cc^*bd$ " (from the removal of q_2).

When q_1 is removed, we combine these paths. The transition from S now directly connects to F, and its label becomes the concatenation of the incoming transition to q_1 and the outgoing transition from q_1 . Therefore, the new and final transition from S to F is " $ea(bd \cup cc^*bd)$ ". State q_1 is thus detached, leaving only the start state S and the final state F, with a single regular expression representing all possible paths from start to finish. This results in the final NFA diagram, which is now a regular expression, as shown in Figure 9.

Incoming	Outgoing
S	F

Table 7: Removal of q_1

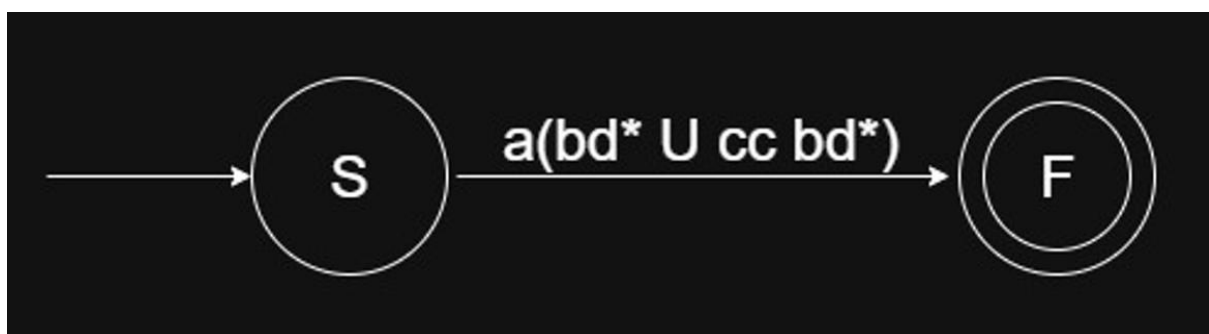


Figure 9: NFA Diagram with removal of q_1

Conclusion Part A

The Non-deterministic Finite Automaton (NFA) designed for the restaurant stock update system effectively models the user's interaction, from initial item entry to successful quantity saving. This NFA, defined by its quintuple and state transition table, clearly illustrates the states (q_0, q_1, q_2, q_3, q_4) and transitions (a, b, c, d) involved in validating user input. The iterative refinement process, involving the systematic removal of states (q_4, q_3, q_2, q_1, q_0) and the addition of start (S) and final (F) states, allowed for the derivation of a regular expression that accurately describes the pattern of valid input sequences. This regular expression captures the logic of ensuring a correct restaurant item is entered, followed by a quantity, before the "Quantity saved" message is displayed, demonstrating the program's defined and predictable flow.

Part B

```
def basic_inventory():  
    restaurant_item = input("\n\tPlease Enter The Restaurant Item: ")  
    while restaurant_item != "Fried Noodles" and restaurant_item != "Cheese pizza" and restaurant_item != "Flat bread" and restaurant_item != "Chicken burger":  
        print("\nThere is no record for Restaurant Item!")  
        restaurant_item = input("\n\tPlease Enter The Restaurant Item: ")  
    quantity = int(input("\n\tPlease Enter The Quantity : "))  
    print("\nQuantity saved!")  
    print("Exit")
```

Figure 10

The given code snippet illustrates a linear search technique. This basic searching method works by sequentially examining each item in a collection, starting from the first, to determine if the target value exists. When the desired element is identified, the algorithm stops and returns its position. If it doesn't find the element, the process continues until all items are checked, at which point it signals that the element isn't present.

In this specific case, the list to be searched isn't explicitly declared. Instead, it's inferred from the user's input. The search operates by determining whether the entered string begins with one of the predefined items such as "Fried noodles," "Cheese pizza," "Flat bread," or "Chicken burger." If a match is found at the start of the string, the search ends, prompting the user to enter a quantity. If no match is found, the program outputs a message stating that the restaurant item isn't recorded.

Linear search operates with a time complexity of $O(n)$, where 'n' represents the total number of items in the list. This indicates that as the list grows, the time to complete the search increases proportionally. Although not the fastest algorithm, its simplicity makes it useful for unsorted or small datasets. In this particular example, the search loop runs at most five times, regardless of input length, making its time complexity constant, or $O(1)$.

1.1 Best-Case Scenario

The optimal outcome occurs when the user input begins with one of the following items: “Fried noodles,” “Cheese pizza,” “Flat bread,” or “Chicken burger.”

1.2 Worst-Case Scenario

In the least favourable scenario, the input does not begin with any of the specified items. As a result, the loop continues indefinitely until a valid item is finally entered.

2.0 Computability

2.1 Complete Computability

To assess whether a piece of code is fully computable, we must evaluate its input, output, and the sequence of operations it uses to transform the input into the output.

In this specific example, the user provides a string as input, and the program returns a message based on whether the string begins with any of the predefined items: “Fried noodles,” “Cheese pizza,” “Flat bread,” or “Chicken burger.” The program processes the input through a loop that runs up to five times. During each cycle, it prompts the user for input, checks if the string matches the expected items at the start, and then displays a corresponding message.

Because the code executes within a limited time frame and follows clear and defined instructions for handling input and producing output, it qualifies as fully computable.

2.2 Mechanism Computability

Mechanistic computability refers to a computer's capability to execute a specific algorithm or program by following a defined set of instructions step by step.

The code snippet provided fits this definition, as it can be executed through a sequence of mechanical operations by a computer. The program uses a simple while loop to request input from the user, check whether the entered string contains one of the predefined items “Fried noodles,” “Cheese pizza,” “Flat bread,” or “Chicken burger” and then displays an appropriate message. Since the loop runs a maximum of five times regardless of input length, the steps are limited and predictable. This shows that the program can be carried out mechanically, making it a clear example of something that is mechanistically computable.

2.3 Deterministic Computability

Deterministic computability refers to a system's ability to produce the same output consistently when given the same input, following a fixed and predictable set of operations. The code snippet in question demonstrates deterministic behaviour, as it reliably produces identical results for repeated instances of the same input. It employs a basic while loop that prompts the user to enter a string, checks if the input begins with specific phrases like “Fried noodles,” “Cheese pizza,” “Flat bread,” or “Chicken burger,” and then displays a corresponding message. Since the program’s outcome is entirely dependent on the input and does not involve any randomness, it will always generate the same response for identical inputs. Thus, the code qualifies as deterministically computable.

Conclusion Part B

Upon reviewing its structure, examples, and concise implementation, it is evident that the program operates with a time complexity of $O(1)$, indicating that its execution duration is determined by a constant number of operations, regardless of input size. The code is fully computable, as it consists of a limited set of instructions, clearly defined input and output, and always concludes after execution. It avoids infinite loops and does not involve any complex or unsolvable logic. Overall, this program represents a basic control flow model and showcases how global variables and functions can be used to simulate a simple restaurant ordering system that updates item quantities.

References

1. Coursera. (2022, November 14). *What is Python used for? A beginner's guide*.
<https://www.coursera.org/articles/what-is-python-used-for-a-beginners-guide-to-using-python>
2. Dataquest. (2025, March). *Python for loop: A beginner's tutorial*.
<https://www.dataquest.io/blog/python-for-loop-tutorial/>
3. GeeksforGeeks. (2023, February 14). *Difference between NFA and DFA*.
<https://www.geeksforgeeks.org/difference-between-dfa-and-nfa/>
4. GeeksforGeeks. (2025, July 23). *Loops in Python – for, while and nested loops*.
<https://www.geeksforgeeks.org/python/loops-in-python/>
5. GeeksforGeeks. (2025, May). *Introduction to theory of computation*.
<https://www.geeksforgeeks.org/theory-of-computation/introduction-of-theory-of-computation/>
6. HackerEarth. (n.d.). *Linear search tutorials & notes | Algorithms*.
<https://www.hackerearth.com/practice/algorithms/searching/linear-search/tutorial/>
7. Medium – Sehgal, K. (2017). *An simplified explanation of linear search*.
<https://medium.com/karuna-sehgal/an-simplified-explanation-of-linear-search-5056942ba965>
8. Programiz. (n.d.). *Linear search algorithm*. <https://www.programiz.com/dsa/linear-search>
9. Real Python. (2025, February). *Python for loops: The Pythonic way*.
<https://realpython.com/python-for-loop/>
10. Studytonight. (2023). *Difference between DFA and NFA*.
<https://www.studytonight.com/difference-between/dfa-vs-nfa>
11. Tutorialspoint. (2023, October 31). *What is the difference between DFA and NFA?*
<https://www.tutorialspoint.com/what-is-the-difference-between-dfa-and-nfa/>
12. Unstop Blog. (2025, February). *What is linear search? Algorithm, working, complexity & examples*. <https://unstop.com/blog/linear-search>
13. W3Schools. (2025). *Python for loops*.
https://www.w3schools.com/python/python_for_loops.asp
14. Wikipedia contributors. (2025, March). *Powerset construction*. Wikipedia.
https://en.wikipedia.org/wiki/Powerset_construction
15. Wikipedia contributors. (2025, July 10). *Computability theory*. Wikipedia.
https://en.wikipedia.org/wiki/Computability_theory